

bzst: Block-Compressed Zstandard

A parallel, seekable, zstd-compatible container format

Working draft 0.1 — 2026-07-14

Editor: Tim Fennell

*Status of this document: **DRAFT**. This is an in-progress design document, not a ratified specification. Field layouts, magic numbers, and integer widths marked provisional are subject to change; see Section 10.*

Abstract. bzst (“beast”¹) is a container format built on the Zstandard frame format ([RFC 8878](#)). It stores a stream as a sequence of independently-compressed blocks so that compression and decompression can be parallelized, and it embeds a self-contained index that maps uncompressed offsets to compressed offsets so that any position can be reached in $O(\log n)$. A bzst file written in the baseline profile is a valid Zstandard archive: `zstd -d` reproduces the original bytes. bzst is a general-purpose compression container; it is deliberately domain-agnostic and defines no semantic (e.g. genomic) indexing or key/value metadata, providing instead a small set of extension points on which derived formats are built.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Relationship to prior work	3
1.3	Design in one picture	3
2	Conventions and terminology	4
3	The bzst skippable-frame namespace	4
4	Header frame (subtype 0x00)	5
5	Block layout: block-header frame and data frame	6
5.1	Block-header frame (subtype 0x01)	6
5.2	Data frame	7
5.3	Parallel decode	7
6	The index (subtype 0x02)	7
6.1	Locating the index	8
6.2	Index frame contents	8
6.3	Optional index compression	9
7	Profiles	9
8	Derived formats	10
8.1	Anchoring a derived index on the bzst index	10
9	Recommendations (non-normative)	10
10	Open issues	11
10.1	Integer width for block sizes	11
10.2	Dictionary profile	11
10.3	Magic-number selection	11
10.4	Per-block codec tagging	11
10.5	Concatenation and oversized indices	11

¹The name is “block-compressed zstd.” Said aloud it is *beast*; homage to the X-Men’s Dr. Henry McCoy — the one who is simultaneously the strongest in the room and the one who read the literature first — is entirely intentional.

10.6	Truncation detection	11
10.7	Block-header checksum	11
11	Resolved decisions and rationale	12
12	OpenZL: findings and forward-compatibility	13
13	Acknowledgements	13
14	References	13
15	Appendix A — Reference tooling wishlist (informative)	13

1 Introduction

1.1 Motivation

The BGZF format has served genomics well as the compression layer beneath BAM, VCF/BCF and Tabix-indexed files, but it is showing its age: it is tied to DEFLATE, it caps blocks at 64 KiB (a limit forced by the 48/16-bit virtual offsets of the BAI and CSI indices, not by anything intrinsic), and its indices are external files with all the attendant naming, staleness and object-store problems. Meanwhile Zstandard offers materially better ratio and much higher throughput than DEFLATE, and its *skippable frame* mechanism provides a clean, standard way to carry format metadata inside an otherwise-ordinary compressed stream.

bzst is a Zstandard-based successor to BGZF with four goals:

Parallelism Both compression and decompression **MUST** be trivially parallelizable, whether the input is a seekable file or a forward-only stream (a pipe).

Seekability A reader **MUST** be able to map an uncompressed byte offset to a compressed location and begin decompressing there, without scanning the whole file.

Self-containment The mapping from uncompressed to compressed space **MUST** live inside the file. No sidecar index is required to random-access the compressed container.

Compatibility A file in the baseline profile **MUST** remain a valid Zstandard stream that any conformant `zstd` decoder can decompress to the original bytes.

bzst is *not* genomics-specific. Although it grew out of a genomics need and its recommendations (Section 9) speak to genomic use cases, the core format knows nothing about records, coordinates, references, or samples.

1.2 Relationship to prior work

bzst draws directly on several existing designs; Section 13 credits them in full. In brief: it uses Zstandard skippable frames the way the `pzstd` and `seekable_format` “contrib” tools do; it adopts the two-layer index idea (uncompressed $\leftrightarrow$ compressed separated from semantic indexing) and the larger-block and self-contained-index goals from J. Bonfield’s **BGZF2** proposal; and it takes lessons on what to fix from the `seekable_format` spec and its `zeekstd` reimplementation.

1.3 Design in one picture

A bzst file is a sequence of Zstandard frames. Structural metadata travels in *skippable* frames (which any `zstd` decoder ignores); payload travels in ordinary *data* frames. Each data frame is immediately preceded by a small skippable *block-header frame* that gives the compressed and uncompressed length of that data frame — this is what makes streaming parallel decode possible without parsing frame internals. A skippable *index frame* at the end maps the file for random access.

```
+-----+
| Header frame          bzst skippable (subtype 0x00) |
+-----+
| Block-header frame    bzst skippable (subtype 0x01) | \
| Data frame           zstd data frame                | /  block 0
+-----+
| [ optional derived-format frames: other skippable magics ] |
+-----+
| Block-header frame    bzst skippable (subtype 0x01) | \
| Data frame           zstd data frame                | /  block 1
+-----+
| ...                  |
+-----+
```

```

| [ optional derived index frame: derived format's own magic ]|
+-----+
| Index frame          bzst skippable (subtype 0x02)      |
|   ... entries ...                                     |
|   [ EOF trailer: index_offset + magic = last bytes of file ]|
+-----+

```

Two invariants make the picture work and are stated normatively later:

1. **Baseline decodability** (Section 7): in the baseline profile every data frame is an ordinary zstd frame, so concatenating the decoded output of all data frames — which is exactly what `zstd -d` does, skipping the skippable frames — yields the original uncompressed stream.
2. **Adjacency** (Section 5): a block-header frame and its data frame are physically adjacent, so the two can always be paired.

2 Conventions and terminology

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are to be interpreted as in RFC 2119.

All multi-byte integers are **unsigned** and **little-endian**, matching the Zstandard frame format and existing genomics tooling. `u8`, `u16`, `u32`, `u64` denote unsigned integers of that width.

Frame A Zstandard frame as defined by RFC 8878: either a *data frame* (magic `0xFD2FB528`) or a *skippable frame* (magic `0x184D2A50–0x184D2A5F`).

Skippable frame A frame carrying an opaque payload that a conformant Zstandard decoder skips. Its wire form is a 4-byte magic, a 4-byte `u32` `Frame_Size`, and `Frame_Size` bytes of user data.

Data frame An ordinary Zstandard (or, in a future profile, other-codec) frame carrying compressed payload.

Block The pair [block-header frame][data frame] — the unit of independent (de)compression and the unit the index addresses.

Structural frame A bzst-defined skippable frame (header, block-header, index, ...). Structural frames all share one reserved skippable magic and are distinguished by a subtype byte (Section 3).

Derived format A higher-level format (e.g. a BAM or BCF encoding) layered on bzst, which uses its own frames — carried under *other* skippable magics — for its own metadata and indices.

Profile A payload-codec capability (Section 7). The *baseline* profile is plain zstd, no dictionary; a file declares the set of profiles it uses.

3 The bzst skippable-frame namespace

Zstandard reserves sixteen skippable magic numbers, `0x184D2A50` through `0x184D2A5F`. This is a small, globally-shared, uncoordinated space (for example `pzstd` conventionally uses `0x184D2A50` and `seekable_format` uses `0x184D2A5E`). bzst therefore claims **exactly one** of these values for all of its structural frames and distinguishes their roles with an internal subtype byte, leaving the other fifteen magics for derived formats and other tools.

Note. Provisional magic: bzst uses `0x184D2A5B`. Final selection (and the choice to avoid the `0x50` and `0x5E` conventions) is Section 10.3.

Every bzst structural frame has this envelope:

Bytes	Field	Value / Type	Description
4	<code>Magic_Number</code>	<code>0x184D2A5B</code>	bzst structural-frame magic (little-endian on disk).

Bytes	Field	Value / Type	Description
4	Frame_Size	u32	Number of bytes that follow (everything below this row).
1	Subtype	u8	Structural-frame role; see registry below.
Frame_Size-1	Payload	bytes	Subtype-specific content, defined per subtype in the following sections.

The bzst **format version** is a whole-file property, declared once in the header frame (Section 4); it is not repeated per frame. A reader **MUST** skip any structural frame whose Subtype it does not recognize. Because structural frames are skippable, an unknown subtype never impairs decompression — only the higher-level function that subtype would have provided. Each structural-frame payload places its fixed fields at the front and **MAY** carry additional trailing fields in later format versions; readers use the fixed leading fields, locate any trailing checksum via Frame_Size, and ignore trailing bytes they do not recognize.

Subtype registry.

Subtype	Name	Purpose
0x00	Header	File signature, format version, profiles. Exactly one, first frame in the file (Section 4).
0x01	BlockHeader	Compressed + uncompressed length of the following data frame, plus per-block flags (Section 5).
0x02	Index	Uncompressed→compressed jump table; last frame in the file (Section 6).
0x03	Dictionary	<i>Reserved.</i> Embedded compression dictionary for the dictionary profile (Section 10.2).
0x04–0xFF	—	Reserved for future bzst use.

Derived formats **SHOULD** place their own metadata in skippable frames using one of the other fifteen skippable magics, and **SHOULD** adopt the same (magic, u32 Frame_Size, ...) envelope shape for consistency. bzst assigns no meaning to those magics and never inspects them.

4 Header frame (subtype 0x00)

The header frame **MUST** be the first frame in the file. It makes the file recognizable as bzst from its first bytes, declares the format version, and declares which profiles the file uses.

Bytes	Field	Value / Type	Description
4	Magic_Number	0x184D2A5B	Structural-frame magic.
4	Frame_Size	u32	Bytes following.
1	Subtype	0x00	Header.
1	Format_Version	0x01	bzst format version; governs interpretation of the whole file.
4	Signature	"BZST"	0x42 0x5A 0x53 0x54. Identifies the container as bzst.
4	Format_Signature	u32 / tag	Opaque 4-byte derived-format tag; 0x00000000 = none (generic bzst). bzst never interprets it (Section 8).
1	Profiles	u8	Bitmask of profiles used anywhere in the file (Section 7). 0x00 = baseline.
1	Flags	u8	Reserved; MUST be 0x00 and MUST be ignored on read.

Bytes	Field	Value / Type	Description
8	Checksum	u64	XXH64 of all preceding bytes of this frame.

Recognition. A file is bzt if its first four bytes are a Zstandard skippable magic and the four bytes at offset 10 are "BZST". A derived format additionally checks the four bytes at offset 14 against its `Format_Signature`; together these form a fixed 8-byte type magic `[BZST][tag]` that a tool such as `file/libmagic` can match. The test is cheap, is robust to the leading skippable frame being ignored by generic tools, and does not collide with a plain `zstd` file (whose first bytes are the data-frame magic `0xFD2FB528`).

Note. `Format_Signature` lets a derived format (say `bzt-bam`) recognize its own files from the header alone, without parsing anything bzt-specific. bzt treats it as opaque: it is an extension point, not domain knowledge. Richer content sniffing and all derived metadata remain the derived format's concern, carried in the derived format's own frames (Section 8).

5 Block layout: block-header frame and data frame

Payload is stored as a sequence of **blocks**. Each block is a **block-header frame** immediately followed by a **data frame**.

Adjacency invariant. A block-header frame and the data frame it describes **MUST** be physically adjacent. No other frame of any kind — structural, derived-format, or data — **MAY** appear between them. (Derived-format frames **MAY** appear before the block-header frame or after the data frame, never between.)

This invariant lets any reader pair a block-header frame with its data frame by adjacency alone, and lets the index address a whole block with a single offset.

5.1 Block-header frame (subtype 0x01)

The block-header frame carries the sizes needed to place and decode the following data frame, plus a flags byte. Its fixed leading fields (below) are stable across format versions; later versions **MAY** append additional per-block metadata before the trailing checksum, which a reader locates via `Frame_Size`.

Bytes	Field	Value / Type	Description
4	Magic_Number	0x184D2A5B	Structural-frame magic.
4	Frame_Size	u32	Bytes following.
1	Subtype	0x01	BlockHeader.
8	Compressed_Size	u64	Exact on-disk size of the following data frame.
8	Uncompressed_Size	u64	Size of the following data frame's decoded output.
1	Flags	u8	Bit 0: Stored (advisory; see Section 5.2). Bits 1–7 reserved (0x0).
4	Checksum	u32	Low 32 bits of XXH64 over all preceding bytes of this frame; the trailing 4 bytes, located via <code>Frame_Size</code> .

TODO. Integer width of `Compressed_Size` / `Uncompressed_Size` (u64 here) is Section 10.1. Whether the block-header checksum should be mandatory, optional (flag-gated), or dropped in favour of the data frame's own content checksum is Section 10.7.

5.2 Data frame

A data frame is a standard Zstandard data frame (RFC 8878), magic 0xFD2FB528. `bzst` adds no constraints beyond those of the active profile (Section 7). The following is an informative summary so implementors need not consult the RFC for routine work; **RFC 8878 is authoritative**.

A `zstd` frame is `Magic_Number` (4 bytes) followed by a `Frame_Header` (2–14 bytes), one or more `Blocks`, and an optional 4-byte `Content_Checksum`.

- **Frame_Header.** A 1-byte `Frame_Header_Descriptor` selects which optional fields follow: the `Frame_Content_Size` (the decoded size; 0/1/2/4/8 bytes), a `Single_Segment` flag, a `Content_Checksum` flag, and a `Dictionary_ID` (0/1/2/4 bytes). A `Window_Descriptor` byte (giving the decode window / memory size) is present unless `Single_Segment` is set.
- **Blocks.** Each block is a 3-byte `Block_Header` plus content. The header packs `Last_Block` (1 bit, marks the final block of the frame), `Block_Type` (2 bits), and `Block_Size` (21 bits). `Block_Type` is `Raw` (0, stored verbatim), `RLE` (1, one byte repeated), `Compressed` (2), or `Reserved` (3).
- **Content_Checksum.** If the descriptor’s checksum flag is set, a trailing 4 bytes carry the low 32 bits of the XXH64 of the decoded content. Writers **SHOULD** enable it so any decoder detects per-block payload corruption.

Store-only data. Zstandard has a native uncompressed mode at the block level (`Block_Type = Raw`), the analogue of DEFLATE’s level-0 stored blocks; `libzstd` emits it for incompressible input. A data frame whose blocks are all `Raw` is effectively uncompressed yet remains a valid `zstd` frame, so baseline decodability (Section 7) still holds. Whether a frame is stored is determinable **only** by parsing its block headers — there is no frame-level “stored” flag — and equal `Compressed_Size` and `Uncompressed_Size` is **not** a reliable signal (framing overhead usually makes a stored frame slightly larger than its content, and a compressed frame’s sizes can coincide). `bzst` therefore offers the advisory `Stored` bit in the block-header `Flags` for writers that want cheap store-only detection; it is a hint only, and the data frame’s framing is authoritative.

5.3 Parallel decode

Streaming (forward-only). A reader loops: read the block-header frame, learn `Compressed_Size` and `Uncompressed_Size`, read exactly `Compressed_Size` bytes for the data frame, and hand the pair (compressed bytes, expected output size) to a worker. No frame internals are parsed to find boundaries, and the output buffer is pre-sized exactly. This is possible on a pipe, with no seeking and no index.

Seekable. A reader loads the index (Section 6) and binary-searches it to find the block covering a target uncompressed offset, then reads and decodes that block.

Note. Because the container learns block boundaries from the block-header frames rather than by parsing data frames, it is **codec-agnostic**: the payload codec can change (Section 7) without changing how blocks are found or the file is indexed. This is the mechanism by which `bzst` is “OpenZL-ready” (Section 7).

6 The index (subtype 0x02)

The index is a jump table from uncompressed offsets to compressed locations. It is the last frame in the file. It is an **accelerator**, not a source of truth: the same information is recoverable by a single forward pass over the block-header frames, so a file with a missing or damaged index is still fully decodable and streamable — only $O(1)$ seeking is lost.

6.1 Locating the index

The final bytes of the file form a fixed trailer that lets a reader find the index frame from the end without scanning:

Bytes	Field	Value / Type	Description
8	Index_Offset	u64	Absolute file offset of the index frame's Magic_Number.
4	EOF_Magic	0x8F92EA5B	Sentinel; the last four bytes of the file.

A reader seeks to EOF - 12, reads these twelve bytes, checks EOF_Magic, then seeks to Index_Offset and parses the index frame. Absence of EOF_Magic signals a truncated or non-bzst file. These twelve bytes are the tail of the index frame's payload (they are within its Frame_Size), so a generic zstd decoder still skips the whole frame.

Note. Provisional EOF_Magic 0x8F92EA5B echoes the seekable_format sentinel family (0x8F92EAB1) while being distinct; final value is Section 10.3. Using an absolute Index_Offset (rather than a distance-from-EOF) is simplest for a single file but interacts with concatenation (Section 10.5).

6.2 Index frame contents

Bytes	Field	Value / Type	Description
4	Magic_Number	0x184D2A5B	Structural-frame magic.
4	Frame_Size	u32	Bytes following.
1	Subtype	0x02	Index.
1	Index_Flags	u8	Bit 0: Entries blob is zstd-compressed. Bits 1–7 reserved (0x0).
8	Entry_Count	u64	Number of blocks (= number of entries).
8	Total_Uncompressed	u64	Sum of all blocks' uncompressed sizes; the end sentinel for search.
varies	Entries	see below	Entry_Count entries; zstd-compressed iff Index_Flags bit 0.
8	Checksum	u64	XXH64 over the uncompressed Entries plus the fixed fields above.
8	Index_Offset	u64	Trailer (see above).
4	EOF_Magic	0x8F92EA5B	Trailer; last bytes of the file.

Each entry, in block order, is 24 bytes:

Bytes	Field	Value / Type	Description
8	Uncompressed_Offset	u64	Uncompressed byte offset at which this block's decoded data begins. (Binary-search key.)
8	Block_Offset	u64	Absolute file offset of this block's block-header frame .
8	Block_Length	u64	On-disk length of [block-header frame + data frame].

This entry shape is deliberate. A single read of Block_Length bytes starting at Block_Offset fetches the block-header frame **and** the whole data frame in one I/O — important on high-latency storage systems (e.g. object

stores, network filesystems) — and that one buffer already contains the `Uncompressed_Size` needed to pre-size the decode buffer. Uncompressed size per block is not stored: it is `Uncompressed_Offset` of the next entry minus this one (and `Total_Uncompressed` closes the last block).

To seek to uncompressed offset x : binary-search `Entries` for the greatest `Uncompressed_Offset` $\leq x$; read `Block_Length` bytes at `Block_Offset`; decode the data frame; skip $x - \text{Uncompressed_Offset}$ bytes into the result.

6.3 Optional index compression

Absolute offsets let a reader binary-search the index directly on the memory-mapped file. Setting `Index_Flags` bit 0 instead stores the `Entries` blob as a zstd frame; a reader then decompresses the entire blob into memory once and searches it there (the absolute offsets remain valid). This trades on-disk searchability for a smaller index and is worthwhile for very large files. The fixed fields (`Entry_Count`, `Total_Uncompressed`, flags) are never compressed, so a reader can always read them without decompressing. Index compression is transparent to zstd -d, which skips the whole frame regardless.

TODO. Whether to also store a total compressed length / file length for stronger truncation detection is Section 10.6. An index that exceeds the 4 GiB skippable `Frame_Size` limit (billions of blocks) would need to span multiple frames; deferred (Section 10.5).

7 Profiles

A **profile** is a payload-codec capability. A file may use more than one, because whether an individual block uses a dictionary or a non-zstd codec is **self-describing in the data frame itself** — the zstd frame header carries (or omits) a `Dictionary_ID`, and the codec is identified by the data frame’s magic. A reader therefore dispatches per block for free, and a derived format **MAY**, for example, dictionary-compress some blocks and not others with no bzst-level per-block field.

The header’s `Profiles` byte is a **bitmask of the capabilities used anywhere in the file**, so a reader can fail fast if it lacks one, without scanning every frame:

Bit	Profile	Meaning
—	Baseline	<code>Profiles == 0x00</code> . Every data frame is a plain zstd frame with no dictionary; zstd -d reproduces the original bytes. MUST be supported by every reader.
0	Dictionary	<i>Reserved</i> . Some data frames are zstd-compressed against an embedded dictionary (Section 10.2).
1	Non-zstd codec	<i>Reserved</i> . Some data frames use a non-zstd codec (e.g. a future OpenZL profile).
2–7	—	Reserved (0x0).

Baseline requirement. A generic bzst stream — one produced for direct consumption as .bzst, not as the substrate of a named derived format — **MUST** set `Profiles == 0x00` in v1. This guarantees that any plain .bzst file is decompressible with standard zstd tooling. Derived formats (which identify themselves via `Format_Signature` and their own frames, and are not expected to be consumed by a generic zstd reader) **MAY** use any profile.

Why profiles, and “OpenZL-ready.” Two desirable capabilities both break plain-zstd -d decodability: a **dictionary** (a zstd frame that references a dictionary fails to decode without it) and **OpenZL** (whose output is not a zstd frame at all — it has its own container magic 0xD7B1A5C0 and requires the OpenZL universal

decoder). Universal decodability therefore cannot be a property of **every** bzst file if these are to be supported; it is instead a guaranteed property of the **baseline** profile. Because the container finds blocks via block-header frames rather than by parsing payload (Section 5), adding a codec later requires **no container change**: a reader dispatches per block on the data frame’s own magic. bzst does not specify the non-zstd codec profile now — OpenZL’s wire format is explicitly pre-1.0 and unstable — but reserves the bit. See Section 12.

8 Derived formats

bzst provides the compression container; a derived format provides meaning. The contract is:

- A derived format **MAY** place any number of skippable frames under magics other than bzst’s, subject only to the adjacency invariant (Section 5). bzst never inspects them. It **MAY** declare a 4-byte `Format_Signature` in the header (Section 4) for detection.
- A derived format that is record-based **SHOULD NOT** split a record across a block boundary; each block should contain a whole number of records. (bzst itself has no notion of a record; this is guidance for derived formats — see Section 9.)
- Semantic indexing — by genomic coordinate, by record number, by name — is a derived-format concern, layered **on top of** the bzst index. The natural design maps semantic keys to **uncompressed** offsets; the bzst index then maps those to compressed locations. This is the two-layer split that decouples semantic addressing from block size and removes BGZF’s 64 KiB ceiling.
- bzst defines **no** generic key/value store, and no JSON/YAML-like structure. A derived format that needs such things defines them in its own frames.

8.1 Anchoring a derived index on the bzst index

A derived format that wants its own trailing index (e.g. a genomic coordinate index) can locate it using the bzst index frame as a structural anchor, requiring no bzst support:

1. Find the bzst index via the EOF trailer (Section 6): read `Index_Offset`.
2. Parse the bzst index. Its last entry gives the end of the last data block, `last_block_end = Block_Offset + Block_Length`.
3. Everything in the gap `[last_block_end, Index_Offset)` is whatever the derived format wrote after the last block and before the bzst index — for example its coordinate index frame(s). Because skippable frames are self-delimiting **forward** (`magic + Frame_Size`), the reader simply reads that gap forward to discover them.

No backward seek or derived back-pointer is required; the derived format need only place its frames in that trailing gap. This is a concrete demonstration of the two-layer split: the derived (semantic) index and the bzst (compression) index coexist, each self-contained, with bzst unaware of the former.

9 Recommendations (non-normative)

Block size Choose block size for the access pattern. Smaller blocks (down to BGZF’s 64 KiB) give finer random-access granularity and, at ≤ 64 KiB **uncompressed**, remain compatible with the 16-bit within-block field of BAI/CSI virtual offsets. Larger blocks (hundreds of KiB to a few MiB) give better ratio and throughput. There is no format-imposed minimum or maximum.

Record alignment For record-based data (BAM, BCF, FASTQ), align blocks to record boundaries so no record straddles a block. bzst’s unbounded block size means even a single very large record (a long read, a wide multi-sample VCF row) can occupy its own block — something BGZF’s 64 KiB limit made impossible.

Checksums Enable Zstandard’s content checksum on data frames.

Heterogeneous data For data whose statistics drift along the file (e.g. concatenated FASTQs, name-sorted or multi-sample data), prefer larger blocks, which capture local context per block. A single static dictionary trained on the head of such a file degrades as the data diverges from its training sample (Section 10.2).

Homogeneous data at small block sizes This is the case a dictionary is for; see Section 10.2 and the prototyping plan there.

Derived-format detection Declare a `Format_Signature` in the header and, if needed, put a richer identification frame (under the format’s own magic) early in the file.

10 Open issues

Tracked design questions. Resolved decisions and their rationale are in Section 11.

10.1 Integer width for block sizes

The block-header frame’s `Compressed_Size` / `Uncompressed_Size` are u64 in this draft. u32 would halve the per-block header and matches `seekable_format/zeekstd` precedent while still allowing 4 GiB blocks; u64 future-proofs and honours “the type is the only limit on block size.” Whole-file **offsets** in the index stay u64 regardless. **Lean:** u64. Open.

10.2 Dictionary profile

Whether to specify the dictionary profile (subtype 0x03, embedded dictionary, `Dictionary_ID` wiring) in v1 or merely reserve it; and if specified, whether to allow a **single** embedded dictionary or **multiple** (one per `Dictionary_ID`, selected per block) to serve banded/drifted data. Key facts established: a `zstd` dictionary is fixed across blocks (each block also uses its own in-block history); cross-block “supplementing” is only possible via prefix chaining, which re-couples blocks and is therefore rejected; multiple dictionaries per file are possible because `Dictionary_ID` is per-frame. **Prototyping plan to motivate this:** compare compressing SAM at 64 KiB blocks, at 1 MiB blocks, and at 64 KiB blocks with a dictionary trained on a random sample of records — to measure how much of the large-block ratio a sampled dictionary recovers at small block sizes. Open.

10.3 Magic-number selection

Final values for the `bzst` structural-frame magic (provisional 0x184D2A5B) and the `EOF_Magic` (provisional 0x8F92EA5B), avoiding the 0x184D2A50 (`pzstd`) and 0x184D2A5E (`seekable_format`) conventions. Open.

10.4 Per-block codec tagging

Per-block dictionary/codec use is self-describing (data-frame `Dictionary_ID` and magic), and the header `Profiles` bitmask enables fast-fail, so no per-block `bzst` profile field is needed. Remaining question: is dispatch on the data-frame magic sufficient, or is an explicit per-block codec tag (e.g. in the block-header `Flags`) wanted for robustness? **Lean:** magic dispatch is enough. Open (minor).

10.5 Concatenation and oversized indices

Concatenating two complete `bzst` files yields a valid `zstd` stream that `zstd -d` decodes correctly but that has two headers and two indices. Define whether tools treat this as one logical stream (re-index) or reject it. Related: an index too large for one 4 GiB skippable frame would need to span multiple frames. Both deferred. Open.

10.6 Truncation detection

Whether the index should additionally store a total compressed length / expected file length so that mid-file truncation (not just a missing trailer) is detectable. **Lean:** yes. Open.

10.7 Block-header checksum

Whether the per-block block-header checksum should be mandatory (as drafted), flag-gated optional, or dropped in favour of the data frame’s own content checksum. A corrupt block-header mis-dispatches a parallel decode, which argues for keeping it. Open.

11 Resolved decisions and rationale

- **Scope.** `bzst` is a generic compression container only — no genomics, no generic key/value or JSON/YAML metadata; derived formats own all semantics via their own frames. *Why:* keeps the base format domain-agnostic and small; semantic indexing is inherently format-specific and belongs a layer up.
- **Universal decodability is a profile property**, guaranteed for the baseline profile, not for every file. A direct `.bzst MUST set Profiles == 0` in v1. *Why:* both dictionaries and OpenZL break plain-zstd -d; scoping universality to a profile preserves the interop promise where it matters.
- **Profiles are a header bitmask of capabilities used; per-block behaviour is self-describing** (data-frame Dictionary_ID / magic), so blocks may mix dictionary/non-dictionary and codecs with no per-block `bzst` field. *Why:* zstd already encodes this per frame; a per-block `bzst` profile would be redundant.
- **Format version is a whole-file property** declared once in the header, not repeated per frame. *Why:* the header is always read first, so per-frame version bought only self-description-in-isolation, which is unnecessary — and it cost a byte per block.
- **Format_Signature.** The header carries a 4-byte opaque derived-format tag so a derived format is detectable from a fixed 8-byte `[BZST][tag] magic` without parsing anything else; `bzst` never interprets it. *Why:* cheap, standard type detection; keeps derived detection out of derived-only frames while leaving all real metadata to the derived layer.
- **One reserved skippable magic, subtyped internally;** the other fifteen are left to derived formats. Unknown subtypes **MUST** be skipped. *Why:* frugal with the scarce 16-value global magic space; fully `bzst`-controlled subtype space; no `bzst`-vs-derived collisions inside a file.
- **BlockHeader (subtype 0x01) rather than “sizing”,** and forward-extensible: fixed leading fields, optional trailing per-block metadata in later versions, trailing checksum located via `Frame_Size`. *Why:* clearer name; the block header is the natural home for future per-block metadata.
- **Advisory Stored flag** for zstd Raw (store-only) data frames; the zstd framing is authoritative and such frames stay baseline-decodable. *Why:* equal sizes are an unreliable signal, and there is no frame-level stored flag in zstd.
- **Native index, not reused seekable_format.** *Why:* reuse would force 32-bit sizes, fragile reconstruct-only offsets, and a dummy entry for every interleaved skippable frame; the interop value (one niche contrib tool) is low.
- **Absolute u64 offsets in the index** (on-disk binary search); **compressible from day one** (flag-gated), default uncompressed.
- **The index addresses the block-header frame and stores [uncompressed_offset, block_offset, block_length].** *Why:* one read fetches block header + data together and carries the uncompressed size needed to pre-size the buffer, with no size duplicated except the block length.
- **Inline block-header frames are the source of truth;** the index is a reconstructible accelerator. A missing/damaged index never breaks the file.
- **Derived indices anchor on the bzst index** (Section 8.1) by forward-reading the trailing gap; no `bzst` support required.
- **Little-endian throughout; XXH64 for structural checksums;** recommend zstd content checksums on data frames. *Why:* speed is irrelevant at our scale, so consistency with zstd and a single hash implementation win.
- **Block size is the writer’s choice**, bounded only by the size field’s type; no min/max imposed.
- **No record straddles a block** — a recommendation at the `bzst` level, a **SHOULD NOT** for record-based derived formats (softened from **MUST NOT**: unforeseen use cases may need otherwise).

12 OpenZL: findings and forward-compatibility

OpenZL (Meta, 2025–) is a graph-based, format-aware compressor by the zstd authors. Findings relevant to bzst, from its paper (arXiv 2510.03203), source, and the genomics discussion in issue #76:

- **Its output is not a zstd frame.** OpenZL frames carry their own magic (0xD7B1A5C0-derived) and require the OpenZL universal decoder; even the “fall back to zstd” path stores a **magicless** zstd block inside an OpenZL frame. There is no mode that emits a standard zstd frame. Hence an OpenZL-payload file cannot be a baseline bzst file.
- **Its win is real on ratio and on compression CPU, but not on decode.** Versus high zstd/xz it typically gives 1.3–2× the ratio while compressing several-fold faster, but decompression is often **slower** than zstd for parse-heavy formats (e.g. BAM $\approx$ 249 vs 526 MB/s) — a caveat for a format whose thesis is fast parallel decode.
- **On genomics it beats BAM/BGZF/zstd but trails CRAM $\approx$ 2×,** and trained compressors overfit their training data. So even with an OpenZL profile, bzst is a BGZF/BAM successor and a superior container, not a CRAM-class ratio competitor.
- **Maturity:** v0.x, wire format explicitly changing, backward-decode guaranteed only “for at least the next several years.”

Consequently bzst reserves the non-zstd-codec profile bit but does not specify it now. Forward-compatibility is achieved structurally: the container is codec-agnostic (Section 5), so when OpenZL stabilizes an OpenZL profile can be added with no change to the header, block-header, or index frames.

13 Acknowledgements

This design builds directly on: the Zstandard format and its pzstd and seekable_format contrib tools (Y. Collet, N. Terrell, and the zstd project); the **BGZF2** proposal and prototype (J. Bonfield, Wellcome Sanger Institute), from which the larger-block, self-contained-index, two-layer-index and record-boundary goals are taken; the zeekstd reimplementation (R. Rosen), whose fixes to seekable_format informed the index design; and OpenZL (Meta), which motivated the codec-agnostic, profile-based structure. BGZF itself (H. Li *et al.*) is the format bzst hopes to succeed.

14 References

- RFC 8878 — Zstandard Compression and the application/zstd Media Type. <https://www.rfc-editor.org/rfc/rfc8878>
- Zstandard project and compression-format doc. <https://github.com/facebook/zstd>
- zstd seekable_format spec. https://github.com/facebook/zstd/blob/dev/contrib/seekable_format/zstd_seekable_compression_format.md
- zeekstd seekable format. <https://github.com/roosen/zeekstd>
- BGZF2 proposal (J. Bonfield). <https://github.com/jkbonfield/htslib/blob/bgzf2/BGZF2.md>
- OpenZL. <https://github.com/facebook/openzl> and paper arXiv:2510.03203. <https://arxiv.org/abs/2510.03203>
- xxHash (XXH64). <https://xxhash.com>

15 Appendix A — Reference tooling wishlist (informative)

A living, non-normative list of tools a reference implementation may want. This is implementation guidance, not part of the format.

- **bzst compress / bzst decompress** — parallel block-wise (de)compression to/from a stream, with a target block size and profile.

- **bzst cat** — concatenate several bzst files into one, recomputing a single jump table (and reconciling headers/profiles) rather than leaving multiple indices (Section 10.5).
- **bzst index** — (re)build or repair the index frame from a forward pass over the block-header frames; validate an existing index against the blocks.
- **bzst info** — dump the header (version, Format_Signature, profiles), block count, total sizes, per-block sizes, and checksum status.
- **bzst extract** — random-access extraction of an uncompressed byte range via the index.
- **bzst verify** — check structural and (where present) data-frame checksums, the EOF trailer, and truncation.
- **bzst rehydrate** — transcode an advanced-profile file (e.g. dictionary) down to the baseline profile for consumption by generic zstd tooling.
- **bgzf2bzst** / **bzst2bgzf** — convert to/from BGZF for migration.